```
var_dump(MY_NULL);
```

```
Output:
string(19) "You can't change me"
int(123)
float(1.23)
array(0) {
}
resource(5) of type (stream)
NULL
```

Once declared, a constant is accessed by using the name we gave it when we called `define()`. We can also use the `const` keyword to declare new constants:

```
const MY_STR = "You can't change me.";
```

If we try to re-declare a constant, we'll get an error:

```
define('MY_CONST', "You can't change me");
define('MY_CONST', "A new value");
```

```
Output:
PHP Warning:  Constant MY_CONST already defined
```

This is not necessarily fatal to your program; a warning is just a warning after all, but the constant value will (logically enough) be unchanged, which might impact code further in. We can avoid this altogether by using the built-in `defined()` function:

```
const SOME_CONSTANT = 'a string';

if (defined('SOME_CONSTANT')) {
    echo "SOME_CONSTANT is already defined" . PHP_EOL;
}

if (!defined('SOME_OTHER_CONSTANT')) {
    define('SOME_OTHER_CONSTANT', true);
}
```

```
Output:
SOME_CONSTANT is already defined
```